Energy Consumption Analyzer (Different Domain)

📌 **Problem Statement**

An energy monitoring system records **daily energy consumption readings (in kWh)** from smart meters.

You are required to analyze and process these readings using NumPy to support validation, statistics, normalization, labeling, and formatting.

📌 **Class Declaration**

class EnergyAnalyzer:

📌 **Functionalities to Implement**

---

1. Create Energy Array

Function Prototype

def create_energy_array(self, readings: list) -> np.ndarray

Example Input

create_energy_array([12.5, 30.0, 45.8])

Expected Output

[12.5, 30.0, 45.8]

Implementation Flow

- Convert list to NumPy array
- Ensure dtype is float
- Return the array

---

2. Validate Energy Array

Function Prototype

def validate_energy_array(self, energy_array: np.ndarray) -> bool

Example Input

validate_energy_array(np.array([50, -10, 30]))

Expected Output

False

Implementation Flow

- Check array is not empty
- Ensure all values are positive
- Return True if valid, else False

---

3. Compute Energy Statistics

Function Prototype

def compute_energy_statistics(self, energy_array: np.ndarray) -> tuple

Example Input

compute_energy_statistics(np.array([10.0, 20.0, 30.0]))

Expected Output

(60.0, 20.0, 30.0)

Implementation Flow

- Calculate Total, Average, Maximum
- Round average to **1 decimal**

- Return (total, average, maximum)

---

4. Reduce High Consumption Readings

Function Prototype

def reduce_high_consumption(self, energy_array: np.ndarray) -> np.ndarray

Example Input

reduce_high_consumption(np.array([80.0, 40.0, 60.0]))

Expected Output

[72.0, 40.0, 54.0]

Implementation Flow

- Convert array to float using astype
- Apply **10% reduction** for values **≥ 50**
- Return updated array

---

5. Label High Consumption Days

Function Prototype

def label_high_consumption(self, energy_array: np.ndarray) -> np.ndarray

Example Input

label_high_consumption(np.array([80.0, 40.0, 60.0]))

Expected Output

["High", "Normal", "Normal"]

Implementation Flow

- Compute mean consumption
- Label "High" if value > mean, else "Normal"
- Return NumPy array of labels

---

6. Format Energy Readings

Function Prototype

def format_energy_readings(self, energy_array: np.ndarray) -> np.ndarray

Example Input

format_energy_readings(np.array([25.5, 60.0]))

Expected Output

["25.50 kWh", "60.00 kWh"]

Implementation Flow

- Convert each value to formatted string
- Format → f"{x:.2f} kWh"
- Return NumPy array

✅ **solution.py (Assessment-Ready Source Code)**

```
import numpy as np
class EnergyAnalyzer:

    def create_energy_array(self, readings: list) -> np.ndarray:
        return np.array(readings, dtype=float)

    def validate_energy_array(self, energy_array: np.ndarray) -> bool:
        if energy_array.size == 0:
            return False
```

```
    return np.all(energy_array > 0)

def compute_energy_statistics(self, energy_array: np.ndarray) -> tuple:
    total = float(np.sum(energy_array))
    average = round(float(np.mean(energy_array)), 1)
    maximum = float(np.max(energy_array))
    return (total, average, maximum)

def reduce_high_consumption(self, energy_array: np.ndarray) -> np.ndarray:
    energy_array = energy_array.astype(float)
    energy_array = np.where(
        energy_array >= 50.0,
        energy_array * 0.9,
        energy_array
    )
    return energy_array

def label_high_consumption(self, energy_array: np.ndarray) -> np.ndarray:
    mean_value = np.mean(energy_array)
    labels = [
        "High" if value > mean_value else "Normal"
        for value in energy_array
    ]
    return np.array(labels)

def format_energy_readings(self, energy_array: np.ndarray) -> np.ndarray:
    formatted = [f"{value:.2f} kWh" for value in energy_array]
    return np.array(formatted)
```

Question Code: Q393-P (Weather Heat Index Analyzer)

📌 **Problem Statement**

A city records **daily Heat Index (HI) values** (range 0–70°C equivalent index).
You are required to build utilities to **load, validate, summarize, categorize, and analyze heatwave streaks** using NumPy.

⚠️ **Class creation is NOT required** (functions only)

📌 **Functionalities to Implement**

1️⃣ **Create Heat Index Array**

Creates a NumPy array from a Python list.

Prototype

def create_heat_array(values: list) -> np.ndarray

Example Input

create_heat_array([28, 35, 42, 55])

Expected Output

[28 35 42 55]

Implementation Flow

- Convert list to NumPy array with int64 dtype
- Return the array

2️⃣ **Validate Heat Index Data**

Checks that array is non-empty and all values are between **0 and 60 inclusive**.

Prototype

def validate_heat_array(arr: np.ndarray) -> bool

Example Input

validate_heat_array(np.array([45, -3]))

Expected Output

False

Implementation Flow

- If array is empty → return False
- If all values are between 0–60 → return True
- Else → False

3️⃣ **Compute Heat Index Statistics**

Return **Average, Standard Deviation, Maximum, Minimum** (rounded to 2 decimals).

Prototype

def compute_heat_stats(arr: np.ndarray) -> tuple

Example Input

compute_heat_stats(np.array([30, 42, 55, 60]))

Expected Output

(46.75, 11.79, 60, 30)

Implementation Flow

- Compute average, std, max, min
- Round average & std to 2 decimals
- Return (average, std, max, min)

4️⃣ Categorize Heat Levels

Map each Heat Index value to a category:

Range	Category
0–30	Comfortable
31–40	Warm
41–50	Hot
51–60	Extreme

Prototype

def categorize_heat(arr: np.ndarray) -> np.ndarray

Example Input

categorize_heat(np.array([28, 38, 45, 58]))

Expected Output

["Comfortable", "Warm", "Hot", "Extreme"]

Implementation Flow

- Use **nested np.where**
- Return NumPy array of strings

5️⃣ Longest Heatwave Streak

Length of the **longest consecutive run where Heat Index ≥ 45**.

Prototype

def longest_heatwave_streak(arr: np.ndarray) -> int

Example Input

longest_heatwave_streak(np.array([40, 46, 48, 44, 50, 52, 53]))

Expected Output

3

Implementation Flow

- Assign arr >= 45 to a variable
- Use current_streak and max_streak
- Loop and count consecutive True
- Return max_streak

✅ solution.py (Assessment-Ready Source Code)

```
import numpy as np

def create_heat_array(values: list) -> np.ndarray:
    return np.array(values, dtype=np.int64)

def validate_heat_array(arr: np.ndarray) -> bool:
    if arr.size == 0:
        return False
    return np.all((arr >= 0) & (arr <= 60))

def compute_heat_stats(arr: np.ndarray) -> tuple:
    avg = round(float(np.mean(arr)), 2)
    std = round(float(np.std(arr)), 2)
```

```
    maximum = int(np.max(arr))
    minimum = int(np.min(arr))
    return (avg, std, maximum, minimum)

def categorize_heat(arr: np.ndarray) -> np.ndarray:
    return np.where(
        arr <= 30, "Comfortable",
        np.where(
            arr <= 40, "Warm",
            np.where(
                arr <= 50, "Hot",
                "Extreme"
            )
        )
    )

def longest_heatwave_streak(arr: np.ndarray) -> int:
    condition = arr >= 45
    max_streak = 0
    current_streak = 0

    for value in condition:
        if value:
            current_streak += 1
            max_streak = max(max_streak, current_streak)
        else:
            current_streak = 0

    return max_streak
```
